Threads
Part II

Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Recall from last class

- A thread is an independent flow of execution within a process
- Creating and managing threads has significantly lower overhead compared to processes
- Unlike processes, threads share the same memory space and do not follow a parent-child hierarchy

Basic POSIX thread management in Linux

- `pthread_create()`
  - Create a new thread with a function to execute
- `pthread_self()`
  - Get the ID of the calling thread
- `pthread_join()`
  - Wait for a specific thread to exit and collect its status
- `pthread_exit()`
  - End the calling thread

Shared data in multithreading

- **Shared memory**
  - Global and heap variables are shared among threads
  - Stack variables are private to a thread, but their address may be shared with other threads
- **A critical section is a portion of code that accesses shared data**
- **If multiple threads execute it simultaneously:**
  - Data races may occur
  - Program behavior becomes undefined
- **Mutual exclusion mechanisms ensure that only one thread executes the critical section at a time**

Example – the need for synchronization

```
int counter = 0;

void* thread_1(void *arg){
  for(int i=0; i<100000; i++)
    counter++;                    /* counter++ is not atomic (load, increment, store) */
  pthread_exit(NULL);
}

void* thread_2(void *arg){
  for(int i=0; i<100000; i++)
    counter++;
  pthread_exit(NULL);
}

int main(){
  …
  pthread_create(&thread1, NULL, thread_1, NULL);
  pthread_create(&thread2, NULL, thread_2, NULL);

  pthread_join(thread1, NULL);
  pthread_join(thread2, NULL);

  /* No guarantee on the correctness of the final value */
  printf("Counter value = %d\n", counter);
}
```

Mutex

- **A mutex ensures mutual exclusion, allowing only one thread to execute a critical section at a time**
- **The mutex has two states: locked or unlocked**
- **Used to protect critical sections through two operations:**
  - **Lock** – acquires the mutex; if it is already locked, the thread blocks until it becomes available
  - **Unlock** –releases the mutex, allowing another thread to acquire it

POSIX mutexes in Linux

- **`pthread_mutex_init()`**
  - Initialize a mutex before it can be used by threads
- **`pthread_mutex_destroy()`**
  - Remove a mutex, releasing system resources
- **`pthread_mutex_lock()`, `pthread_mutex_trylock()`, `pthread_mutex_timedlock()`**
  - Acquire a mutex when entering the critical section
- **`pthread_mutex_unlock()`**
  - Release a mutex when leaving the critical section

The `pthread_mutex_init` function

```
#include <pthread.h>

int pthread_mutex_init(pthread_mutex_t *mutex,
                       pthread_mutexattr_t *attr)
```

- **Initializes a mutex for thread synchronization referenced by `mutex`**
- **The `attr` parameter specifies the attributes of the mutex**
  - Use `NULL` for default values
- **Returns:**
  - 0, on success
  - Non-zero value, indicating the error if initialization fails

The `pthread_mutex_destroy` function

```
#include <pthread.h>

int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

- **Releases resources associated with a mutex identified by `mutex`**
  - The mutex can be reinitialized with `pthread_mutex_init()`
- **Operations on a destroyed mutex result in undefined behavior**
- **Returns:**
  - 0, on success
  - Non-zero value, indicating the error if release fails

The `pthread_mutex_lock` function

```
#include <pthread.h>

int pthread_mutex_lock(pthread_mutex_t *mutex);
```

- **Attempts to lock the mutex object referenced by `mutex`**
- **If the mutex is already locked by another thread, the calling thread will block until the mutex becomes available**
  - The thread is suspended by the scheduler and does not consume CPU
- **Returns:**
  - 0, on success
  - Non-zero value, indicating the error if locking fails

The `pthread_mutex_*lock` functions

```
#include <pthread.h>

int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_timedlock(pthread_mutex_t *mutex,
                            const struct timespec *abstime);
```

- **`pthread_mutex_trylock()`**
  - If the mutex object referenced by `mutex` is currently locked (by any thread, including the current thread), the call returns immediately (`EBUSY`)
- **`pthread_mutex_timedlock()`**
  - Enables threads to set timeouts when attempting to acquire a mutex, avoiding deadlocks and indefinite blocking

The `pthread_mutex_unlock` function

```
#include <pthread.h>

int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

- **Unlocks the mutex referenced by `mutex`, allowing other threads to acquire it**

- **Only the thread that locked the mutex can unlock it**
  - Unlocking an already unlocked mutex results in an error
  - Critical distinction from semaphores!

- **Returns:**
  - 0, on success
  - Non-zero value, indicating the error if unlocking fails

Example – using mutexes

- **Mutex lifecycle**
  - A mutex must be initialized before it can be used by threads
  - Threads can then lock and unlock the mutex to protect shared data
  - After all threads have finished, the mutex should be destroyed

```
int counter = 0;
pthread_mutex_t mux;

int main(){
  …
  pthread_mutex_init(&mux, NULL);
  pthread_create(&thread1, NULL, thread_1,NULL);
  pthread_create(&thread2, NULL, thread_2,NULL);

  pthread_join(thread1,NULL);
  pthread_join(thread2,NULL);
  pthread_mutex_destroy(&mux);
  printf("Counter value = %d\n", counter);
}
```

Example – using mutexes

- **Critical section**
  - A thread must acquire the mutex before accessing shared data
  - If the mutex is already locked, the thread blocks until it becomes available
  - Once the mutex is acquired, the thread can safely enter the critical section
  - After completing the critical section, the thread must release the mutex

```
void* thread_1(void *arg){
  pthread_mutex_lock(&mux);
  counter++;
  pthread_mutex_unlock(&mux);
  pthread_exit(NULL);
}

void* thread_2(void *arg){
  pthread_mutex_lock(&mux);
  counter++;
  pthread_mutex_unlock(&mux);
  pthread_exit(NULL);
}
```

Exercise 1

- **Calculate the sum of all values of an array, with `N_VEC` positions, using `N_T` threads**

- **Each thread operates on `N_VEC/N_T` positions, adding these to a global sum, to be presented at the end by the main thread**
  - Note 1: be careful with data coherency
  - Note 2: be careful that `N_VEC` may not be a multiple of `N_T`
  - Note 3: try to reduce lock contention on your solution

Condition variables

- **Allow threads to synchronize based on the state of shared data**
  - Threads can wait until a specific condition becomes true
- **Difference from mutexes**
  - Mutexes protect access to shared resources (mutual exclusion)
  - Condition variables allow threads to wait for changes in shared state
- **Important**
  - The programmer must define and check the condition explicitly
  - Condition variables only provide the mechanism to suspend and wake threads
  - They are always used together with a mutex protecting the shared data

POSIX condition variables in Linux

- **`pthread_cond_init()`**
  - Initialize a condition variable before it can be used by threads
- **`pthread_cond_destroy()`**
  - Remove a condition variable, releasing system resources
- **`pthread_cond_wait(),pthread_cond_timedwait()`**
  - Block on a condition variable
- **`pthread_cond_signal(),`**
  `pthread_cond_broadcast()`
  - Signal the condition variable

The `pthread_cond_init` function

```
#include <pthread.h>

int pthread_cond_init(pthread_cond_t *cond,
                      pthread_condattr_t *attr)
```

- **Initializes a condition variable for thread synchronization referenced by `cond`**
- **The `attr` parameter specifies the attributes of the conditional variable**
  - Use `NULL` for default values
- **Returns:**
  - 0, on success
  - Non-zero value, indicating the error if initialization fails

The `pthread_cond_destroy` function

```
#include <pthread.h>

int pthread_cond_destroy(pthread_cond_t *cond);
```

- **Releases resources associated with a condition variable referenced by `cond`**
  - The condition variable can be reinitialized with `pthread_cond_init()`
- **Operations on a destroyed condition variable result in undefined behavior**
- **Returns:**
  - 0, on success
  - Non-zero value, indicating the error if release fails

The `pthread_cond_wait` function

```
#include <pthread.h>

int pthread_cond_wait(pthread_cond_t *cond,
                      pthread_mutex_t *mutex);
```

- **Blocks the calling thread until the condition variable referenced by `cond` is signaled**
  - The thread **blocks** and automatically **releases the associated mutex** referenced by `mutex`
  - Other threads can now access the condition variable and shared resources
  - Once the condition is satisfied and the thread is **awakened**, the **mutex must be** automatically **reacquired** before the function returns
- **Returns:**
  - 0, on success
  - Non-zero value, indicating the error if suspension fails

The `pthread_cond_timedwait` function

```
#include <pthread.h>

int pthread_cond_timedwait(pthread_cond_t *cond,
                           pthread_mutex_t *mutex,
                           const struct timespec *abstime);
```

- **An error is returned (`ETIMEDOUT`) if the absolute time specified by `abstime` passes before the condition `cond` is signaled**

- **Returns:**
  - 0, on success
  - Non-zero value, indicating the error if suspension fails

The `pthread_cond_signal` function

```
#include <pthread.h>

int pthread_cond_signal(pthread_cond_t *cond);
```

- **Wakes at least one thread waiting on the condition variable referenced by `cond`**
  - If no threads are blocked, the signal has no effect (notifications are not queued)
  - If multiple threads are blocked, the system does not guarantee which thread is notified
  - It may wake more than one in some implementations, but that is rare

- **Returns:**
  - 0, on success
  - Non-zero value, indicating the error if suspension fails

The `pthread_cond_broadcast` function

```
#include <pthread.h>

int pthread_cond_broadcast(pthread_cond_t *cond);
```

- **Wakes up all threads blocked on the condition variable referenced by `cond`**
  - If no threads are blocked, the signal has no effect (notifications are not queued)
  - When multiple threads are blocked on a condition variable, the **scheduler determines the order** in which they resume execution after being released

- **Returns:**
  - 0, on success
  - Non-zero value, indicating the error if suspension fails

Lost wakeup problem

- **This is wrong!**
  - The two threads are manipulating `elements` concurrently
  - Notification **can be lost**

```
void* thread_1(void *arg){
   …
   if(elements == 0)
       pthread_cond_wait(&cond_var, ...);
   …
}

void* thread_2(void *arg){
   …
   elements ++;
   if(elements > 0)
     pthread_cond_signal(&cond_var);
   …
}
```

Lost wakeup problem

- (1) Thread 1 checks `elements` is zero, so condition is true
- (2) Thread 2 increments `elements` and (3) signals the condition, but this is lost because Thread 1 has not yet executed `cond_wait()`
- (4) Thread 1 executes `cond_wait()` and blocks (indefinitely)

```
void* thread_1(void *arg){
   …
   if(elements == 0)                    (1)
       pthread_cond_wait(&cond_var, ...);   (4)
   …
}

void* thread_2(void *arg){
   …
   elements ++;                         (2)
   if(elements > 0)
     pthread_cond_signal(&cond_var);     (3)
   …
}
```

Lost wakeup problem

- **A condition variable must always be used together with a mutex protecting the shared state**

```
void* thread_1(void *arg){
  …
  pthread_mutex_lock(&mux);
  if(elements == 0)
    /* This is why the mutex is needed! */
    pthread_cond_wait(&cond_var, &mux);
  pthread_mutex_unlock(&mux);
  …
}

void* thread_2(void *arg){
  …
  pthread_mutex_lock(&mux);
  elements ++;
  if(elements > 0)
     pthread_cond_signal(&cond_var);
  pthread_mutex_unlock(&mux);
  …
}
```

Using condition variables

- **The mutex must be automatically released when `pthread_cond_wait()` blocks**
  - Allowing other threads (e.g., Thread 2) to access the critical section, modify shared data, and signal the condition variable

- **Once signaled, the blocked thread wakes up and must reacquire the mutex before continuing execution**
  - Ensuring consistency within the critical section

If vs While

- **After a thread is released from `pthread_cond_wait()`, it must retest the condition that led to blocking**
  - Not just when entering the critical section!

- **Main reasons:**
  - Although not very common, the OS may unblock a `pthread_cond_wait()` without a signal (aka spurious wakeups)
  - Other threads may modify shared data between signaling and mutex reacquisition

- **Rechecking ensures that threads proceed only when conditions are truly met**
  - Preventing logical errors, unexpected behavior, or crashes

If vs While

- The condition may no longer be true in threads executing `thread_1()` after wakeup and mutex reacquisition

```
void* thread_1(void *arg){
  …
  pthread_mutex_lock(&mux);
  if(elements == 0)
    pthread_cond_wait(&cond_var, &mux);
  pthread_mutex_unlock(&mux);
  …
}

void* thread_2(void *arg){
  …
  pthread_mutex_lock(&mux);
  elements ++;
  if(elements > 0)
     pthread_cond_signal(&cond_var);
  pthread_mutex_unlock(&mux);
  …
}
```

If vs While

- Always use a `while` loop around `pthread_cond_wait()` to revalidate conditions before proceeding

```
void* thread_1(void *arg){
  …
  pthread_mutex_lock(&mux);
  while(elements == 0)
    pthread_cond_wait(&cond_var, &mux);
  pthread_mutex_unlock(&mux);
  …
}

void* thread_2(void *arg){
  …
  pthread_mutex_lock(&mux);
  elements ++;
  if(elements > 0)
     pthread_cond_signal(&cond_var);
  pthread_mutex_unlock(&mux);
  …
}
```

Exercise 2

- **Create `N_T` worker threads that increment a shared counter by a random value between 1 and 3**
- **When the counter is greater or equal to `MAX`, a printer thread should be notified**
  - All the worker threads end their execution
- **The printer thread passively waits for the notification and prints the value of the shared counter**